Research Square

Preprints are preliminary reports that have not undergone peer review. They should not be considered conclusive, used to inform clinical practice, or referenced by the media as validated information.

A new deep neural network for forecasting: Deep dendritic artificial neural network

Erol Egrioglu
Giresun University
Eren Bas
eren.bas@giresun.edu.tr

Giresun University

Research Article

Keywords: Forecasting , Deep artificial neural networks , Differential evolution algorithm , Dendritic neuron model

Posted Date: May 12th, 2023

DOI: https://doi.org/10.21203/rs.3.rs-2913556/v1

License: This work is licensed under a Creative Commons Attribution 4.0 International License. Read Full License

Additional Declarations: No competing interests reported.

Version of Record: A version of this preprint was published at Artificial Intelligence Review on June 11th, 2024. See the published version at https://doi.org/10.1007/s10462-024-10790-7.

A new deep neural network for forecasting: Deep dendritic artificial neural network

Erol Egrioglu[1]. Eren Bas[1]

Abstract

Deep artificial neural networks have become a good alternative to classical forecasting methods in solving forecasting problems. Popular deep neural networks classically use additive aggregation functions in their cell structures. It is available in the literature that the use of multiplicative aggregation functions in shallow artificial neural networks produces successful results for the forecasting problem. A type of high-order shallow artificial neural network that uses multiplicative aggregation functions is the dendritic neuron model artificial neural network, which has successful forecasting performance. The first contribution of this work is the transformation of the dendritic neuron model, which works with a single output in the literature, into a multi-output architecture. The second contribution is to propose a new dendritic cell based on the multi-output dendritic neuron model for use in deep artificial neural networks. The other most important contribution of the study is to propose a new deep artificial neural network, a deep dendritic artificial neural network, based on the dendritic cell. The training of the deep dendritic artificial neural network is carried out with the differential evolution algorithm. The forecasting performance of the deep dendritic artificial neural network is compared with basic classical forecasting methods and some recent shallow and deep artificial neural networks over stock market time series. As a result, it has been observed that deep dendritic artificial neural network produces very successful forecasting results for the forecasting problem.

Keywords Forecasting . Deep artificial neural networks . Differential evolution algorithm . Dendritic neuron model.

1 Introduction

In the world we live in, we always need to make accurate forecasts to plan the future. This is called a time series forecasting problem within the scope of statistical science. In the solution of the time series forecasting problem, the past movements and realizations of the time series are taken into account and tried to be explained by statistical models. Classical forecasting methods are usually based on linear models for lagged variables of the time series. Artificial neural networks can successfully solve the forecasting problem by utilizing a combination of flexible nonlinear functions and using lagged variables as inputs. Multilayer perceptron (MLP) artificial neural networks proposed by Rumelhart et al. (1986) are the most frequently used shallow artificial neural network type in the literature for solving the forecasting problem. While MLP is based only on the additive aggregation function, artificial neural networks based on the multiplicative aggregation function have also been proposed in the literature.

The single multiplicative neural cell artificial neural network method proposed by Yadav et al. (2007) can solve the

✉ Eren Bas
eren.bas@giresun.edu.tr
Erol Egrioglu
erol.egrioglu@giresun.edu.tr
[1]Giresun University, Faculty of Arts and Science, Department of Statistics,
28200, Giresun, Türkiye.

prediction problem with a single neuron as successfully as MLP. This has led to the investigation of artificial neural networks based on the multiplicative aggregation function and many artificial neural networks using the multiplicative aggregation function have been proposed in the literature. Shin and Gosh (1991) proposed an artificial neural network called Pi-Sigma artificial neural network, which is similar to MLP but different from MLP, using a multiplicative aggregation function in the output layer. In the Pi-Sigma neural network, the weights in the hidden layer and the output layer are taken as fixed, whereas in Egrioglu and Bas (2023), where these weights are taken as a variable, it is shown that improvement is provided for the solution of the forecasting problem compared to the Pi-Sigma ANN. Another artificial neural network that uses the multiplicative aggregation function is the Sigma-Pi artificial neural network, which has been developed by Rumelhart and McClelland (1988) and Gurney (1989). The training of the Sigma-Pi artificial neural network with the grey wolf optimization algorithm was proposed in Sarıkaya et al. (2023) and its applications on the forecasting problem were performed. Another artificial neural network using the multiplicative neuron model is the dendritic neuron model artificial neural network (DNM-ANN) proposed in Todo et al. (2014), which takes different nonlinear transformations of all raw inputs and works on the transformed inputs, thus including the data augmentation process in the process. The dendritic neuron model artificial neural network has been used in many studies in the literature to obtain time series forecasting. In Zhou et al. (2016), DNM-ANN is applied to the forecasting problem of financial time series. In Song et al. (2019), a modification of DNM is presented together with a new training algorithm. In He et al. (2021), the time series were decomposed by the seasonal trend decomposition method and the method of obtaining forecasts over the decomposed series with DNM-ANN was preferred. In Tang et al. (2021), a training algorithm based on the Artificial immune system optimization algorithm was used to train the DNM-ANN. In Tang et al. (2022), the dendritic neuron model for stock price movement forecasting is modified. In Al-Qaness et al. (2022), DNM-ANN was used for crude-oil-production forecasting. In Yılmaz and Yolcu (2022) and Yılmaz and Yolcu (2023), particle swarm optimization-based training algorithms for DNM-ANN were proposed. In Nayak et al. (2022), an improved chemical reaction optimization algorithm-based dendritic neuron model is proposed for financial time series forecasting. In Gul et al. (2023), a statistical learning method for DNM-ANN was proposed for the first time in the literature based on various probability distributions. In Egrioglu et al. (2022), a recurrent dendritic neuron model artificial neural network was proposed for the first time in the literature. The proposed artificial neural network in Egrioglu et al. (2022) has a structure in which the error of the network is feedback. Although Egrioglu et al. (2022) produce successful prediction results, it is not a deep neural network and does not have the advantage of increasing the number of hidden layers for more successful modeling. In Egrioglu et al. (2023), a robust ANN using a dendritic neuron model and its robust learning algorithm is proposed.

Deep artificial neural networks, which have very useful results, especially in the field of image processing, have started to be preferred in solving the forecasting problem in recent years. Recurrent deep neural networks such as long short-term memory (LSTM) and gated recurrent unit (GRU) have been the most frequently used deep neural networks in the field of forecasting thanks to their structure using timesteps. A summary of the literature on deep neural networks can be given as follows. Tian et al. (2018) used LSTM and convolutional neural network (CNN) methods for load forecasting. Veeramsetty et al. (2021) also carried out a study on load forecasting using factor analysis and LSTM. Ning et al. (2022) compared the performance of ARIMA, LSTM, and Prophet methods for oil production forecasting. Karasu and Altan (2022) used the LSTM method for oil time series prediction. Bilgili et al. (2022) performed electricity energy consumption forecasting using LSTM. Liu et al. (2022) proposed a new deep-learning forecasting method for satellite network traffic prediction by developing the GRU artificial neural network.

The motivation of this study is to contribute to the solution of the forecasting problem by proposing a deep recurrent artificial neural network using a dendritic neuron model for the forecasting problem. The contributions of the work are presented in the following sentences. In this study, a deep recurrent artificial neural network using the dendritic neuron model is proposed, which has started to achieve successful forecasting results in recent years. This proposed new deep recurrent artificial neural network is named a Deep dendritic artificial neural network (DeepDenT). To create the DeepDenT deep artificial neural network, a new "dendritic cell" structure is created. Dendritic cells, just like LSTM and GRU cells, can work like a mini neural network that can receive many inputs and produce many outputs. For the generation of the dendritic cell, the

DNM-ANN proposed by Todo et al. (2014) is modified into a multivariate dendritic neuron model (MDNM). DeepDenT is designed to incorporate the proposed new architecture consisting of a hierarchical arrangement of dendritic cells. A training algorithm based on the differential evolution optimization method is proposed for training the DeepDenT neural network. The proposed training algorithm can get rid of local optimum traps more easily thanks to the restart strategy it contains and can find a solution to overfitting problems thanks to the early stopping condition. Since the proposed training algorithm does not require derivatives of the objective function, it does not involve exploding or vanishing gradient problems such as LSTM.

In the second part of the study, the newly proposed MDNM artificial neural network will be introduced. In the third part of the study, "dendritic cell" will be introduced. In the fourth section, DeepDenT artificial neural network and its training algorithm will be introduced. In the fifth section, applications of stock market time series and comparison results with other methods in the literature will be presented. In the last section, the advantages, improvements, and limitations of DeepDenT artificial neural network will be discussed by considering the findings obtained in the application.

2 MDNM artificial neural network

DNM-ANN is proposed by Todo et al. (2014) in a multi-input and single-output structure and can be used to obtain the forecasting of a single time series. In this section, the structure of the DNM-ANN has been made multi-output to allow it to form a cell structure in a deep neural network, and the formulas for calculating the output of the network and the architecture of the MDNM are introduced. The architecture of the MDNM artificial neural network is given in Fig. 1. As can be seen from the figure, since more than one neuron is used in the output layer of the network, the number of parameters has to increase twice the number of additional outputs. In addition, for the training of such a network, the total error for all outputs must be taken into account. The training problem of this network is out of the scope of this study because the reason why we propose this network is that it is used in the construction of the "dendritic cell" structure.

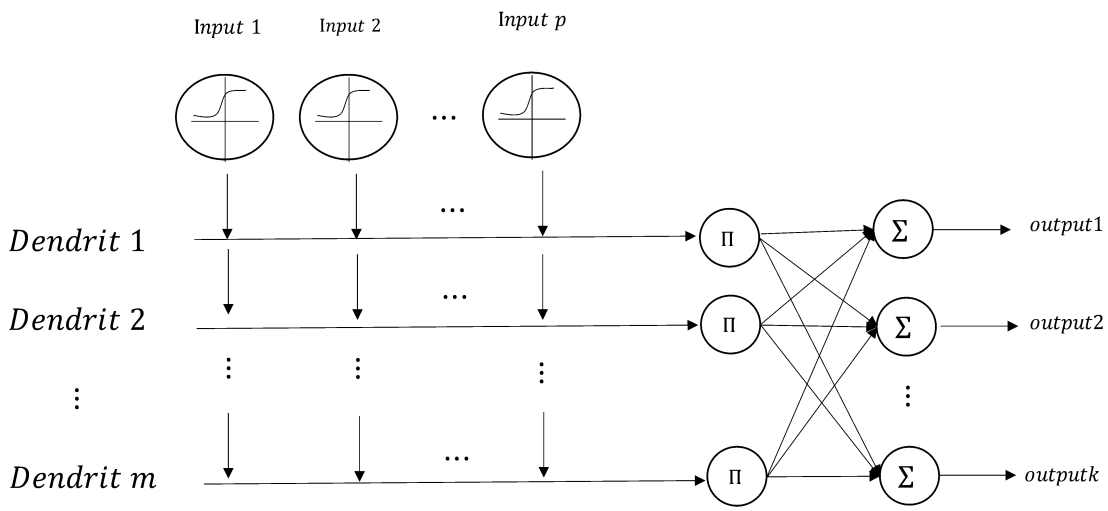

Fig. 1 The architecture of the MDNM artificial neural network

What is important for this study is how to generate the outputs of the MDNM neural network for a given input set. The output of MDNM is given by the following equations. Synaptic functions for an MDDNM with p-input, m dendrites, and k outputs are calculated as in Equation (1).

$$Y_{ij} = \frac{1}{1+\exp\left(-k\left(w_{ij}Input_i+\theta_{ij}\right)\right)} \quad i = 1,2,\dots,p; j = 1,2,\dots,m \tag{1}$$

Dendritic functions are calculated by multiplying synaptic functions as in Equation (2).

$$Z_j = \left(\prod_{i=1}^{p} Y_{ij}\right); j = 1,2,\dots,m \tag{2}$$

Membrane functions are calculated by summing the dendritic functions as in Equation (3). Finally, the output of the network is calculated as in Equation (4).

$$V = \sum_{j=1}^{m} Z_j \tag{3}$$

$$Output_l = \frac{1}{1+exp(-k_{soma}^{l}(V-\theta_{soma}^{l}))} \quad l = 1,2,\dots,n \tag{4}$$

The total number of parameters in the MDNM artificial neural network is $2pm + 2k + 1$ and the parameters of the network are given in Table 1 with the number of elements it contains.

Table 1 The parameters (weights and biases) of MDNM

Parameter	Number
w_{ij} ($i = 1,2,\dots,p;\ j = 1,2,\dots,m$)	$p \times m$
θ_{ij} ($i = 1,2,\dots,p;\ j = 1,2,\dots,m$)	$p \times m$
k	1
k_{soma}^{l} ($l = 1,2,\dots,k$	k
θ_{soma}^{l}($l = 1,2,\dots,k$):	k

k : Slope parameter for synaptic function, k_{soma}^{l} :Slope parameters for outputs transformations, θ_{soma}^{l}: centralization parameter for output transformation

3 Dendritic cell

To create the DeepDenT deep artificial neural network, a new "dendritic cell (DnC)" structure is created. DnC, just like LSTM and GRU cells, can work like a mini neural network that can receive many inputs and produce many outputs. The architectural structure of the DnC is given in Fig. 2.

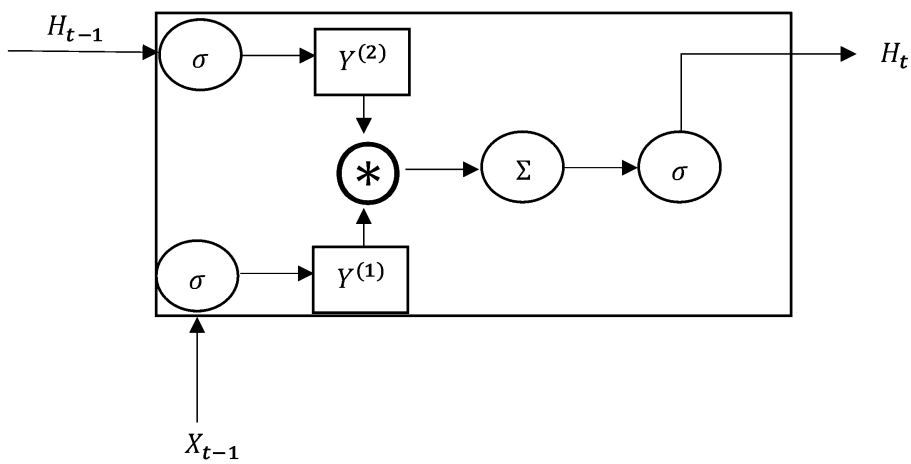

Fig. 2. The architecture of DnC

In a DnC, given feature number p, number of hidden layer units h and dendrite number m, the output of the DnC is calculated by the following equations.

$$Y_{ij}^{1} = \frac{1}{1+\exp\left(-k\left(w_{ij}x_{t-i}+\theta_{ij}\right)\right)} \quad i = 1,2,\dots,p; j = 1,2,\dots,m \tag{5}$$

$$Y^{(1)} = \sigma(k(W_x \odot X_{t-1} + \theta_x)) \tag{6}$$

In Equation (6), $Y^{(1)}$ represents the synaptic function values calculated for the inputs. In Equation (6), $\sigma(.)$ represents the logistic activation function.

$$Y_{kj}^{2} = \frac{1}{1+\exp\left(-k\left(w_{ij}H_{t-1}+\theta_{ij}\right)\right)} \quad k = 1,2,\dots,h; j = 1,2,\dots,m \tag{7}$$

$$Y^{(2)} = \sigma(k(W_h \odot H_{t-1} + \theta_h)) \tag{8}$$

In Equation (8), $Y^{(2)}$ represents the synaptic function values calculated for the recurrent connections.

In Equations (6) and (8), $\odot$ represents the elementwise multiplication operation. An example is given as follows for this product:

$$\begin{bmatrix} a & d \\ b & e \\ c & f \end{bmatrix} \odot \begin{bmatrix} g & h \end{bmatrix} = \begin{bmatrix} a \times g & d \times h \\ b \times g & e \times h \\ c \times g & f \times h \end{bmatrix} \tag{9}$$

The dendritic function value is calculated by using Equations (10) or (11).

$$Z_j = \left(\prod_{i=1}^{p} Y_{ij}^{1}\right)\left(\prod_{k=1}^{h} Y_{kj}^{2}\right); j = 1,2,\dots,m \tag{10}$$

$$Z = Y^{(1)} \circledast Y^{(2)} \tag{11}$$

$\circledast$ represents the multiplication of cumulative product for columns' elements. An example is given as follows for this product:

$$\begin{bmatrix} a & d \\ b & e \\ c & f \end{bmatrix} \circledast \begin{bmatrix} g & h \\ i & j \end{bmatrix} = \begin{bmatrix} a \times b \times c \times g \times i & d \times e \times f \times h \times j \end{bmatrix} \tag{12}$$

The membrane function value is calculated as in Equation (13).

$$V = \sum_{j=1}^{m} Z_j \tag{13}$$

The elements of the output of the DnC are calculated by using Equation (14).

$$Output_l = \frac{1}{1+exp\left(-k_{soma}^{l}(V-\theta_{soma}^{l})\right)} \quad l = 1,2,\dots,n \tag{14}$$

$$H_t = [Output_1\ Output_2\ \dots\ Output_n] \tag{15}$$

$$H_t = \sigma(-k_{soma} \times V - \theta_{soma}) \tag{16}$$

The number of parameters is $2pm + 2k + 1$ in a DnC. The dimensions of weights and biases in a dendritic cell are listed in Table 2.

Table 2 The parameters (weights and biases) of MDNM

Parameter	Number
W_x	$p \times m$
θ_x	$p \times m$
W_h	$h \times m$
θ_h	$h \times m$
k_{soma}	$1 \times h$
θ_{soma}	$1 \times h$
k	1

The dimensions of calculated vectors and matrices in a DnC are given in Table 3.

- $H_t : X_{t-1} : 1 \times p$
- $Y^{(1)} : p \times m$
- $Y^{(2)} : h \times m$
- $Z : 1 \times m$
- $V : 1 \times 1$

Table 3 The dimensions of calculated vectors and matrices

Calculated Vectors	Dimensions
H_t	$1 \times h$
$Y^{(1)}$	$p \times m$
$Y^{(2)}$	$h \times m$
Z	$1 \times m$
V	1×1

4 DeepDenT artificial neural network and its training algorithm

The DeepDenT artificial neural network is a deep recurrent artificial neural network that combines DnCs. In the output layer of DeepDenT, there is a classical fully connected (FC) layer based on the additive aggregation function. DeepDenT is a partially connected artificial neural network with DnCs in a sequential and hierarchical structure. The architectural structure of DeepDenT is given in Fig. 3.

The DnC given by the dark cell wall is the last cell calculated before the output. The input of a DnC in the DeepDent is lagged variable of $x_t = (y_t, y_{t-1,\ldots,} y_{t-p+1})$ according to the number of hidden layer nodes in Fig. 3. The input for the DeepDent cell in the lower left corner of the architecture is $x_{t-\mathrm{h}} = (y_{t-\mathrm{h}-1}, y_{t-h-2,\ldots,} y_{t-h-p})$ in Fig. 3. The output of the DeepDent deep recurrent artificial neural network is one step ahead forecast of the time series. The architecture in Fig. 3 has h time steps, q hidden layers, m dendrite number, and p inputs or features. The number of neurons in all hidden layers is equal to h. The weight and bias values of all DeepDent cells in the same hidden layer are taken equally. This parameter sharing both reduces the number of parameters and enables a common DeepDent cell that presents the same mathematical model in all time steps. These weights and bias values change in different hidden layers, that is, increasing the number of hidden layers increases the number of parameters of the network, while the number of time steps is not effective on the number of parameters like LSTM and GRU. The output of the DeepDenT is calculated with the following formulas. The parameters of DeepDenT for a hidden layer are combined into a single parameter set given in Equation (17) for ease of illustration. As can be seen in Equation (17), while the parameters change from hidden layer to hidden layer, they do not change for the same hidden layer units, i.e. time

steps, and parameters are shared.

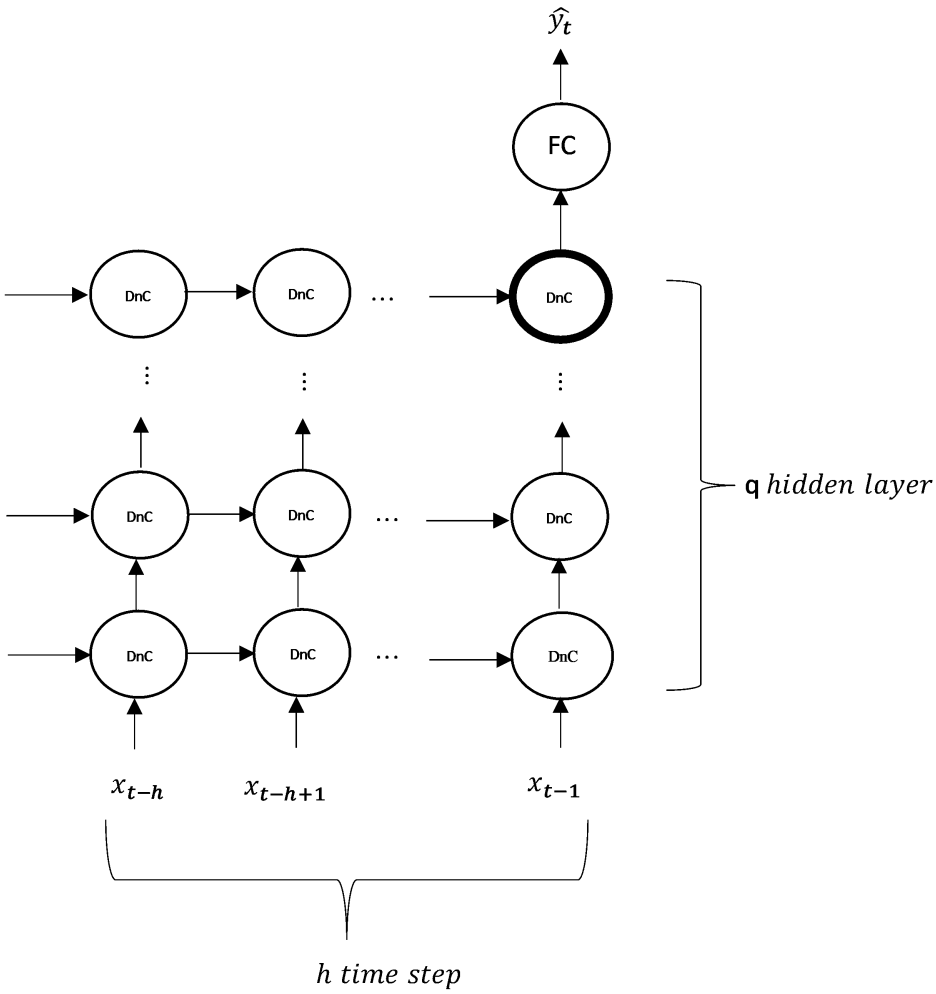

Fig. 3. The architecture of DeepDenT

$$\Theta^j = \left[W_x{}^j, \theta_x^j, W_h{}^j, \theta_h^j, k_{soma}^j, \theta_{soma}^j, k^j\right), j = 1,2, \dots, q \tag{17}$$

The output of the first hidden layer of DeepDenT is calculated by Equation (18). Here, the function f is a representative function representation and the calculations are performed with the formulas given in the DnC section.

$$h_{t-k}^1 = f(h_{t-k-1}^1, x_{t-k}, \Theta^1), k = 1,2, \dots, h \tag{18}$$

In (18), In this equation, h_{t-k}^1 indicates the output obtained in the first hidden layer for the kth time step at time t. Starting from the second hidden layer, the calculations are performed with Equation (19) until the output of DeepDenT is obtained.

$$h_{t-k}^j = f\left(h_{t-k-1}^j, h_{t-k}^{j-1}, \Theta^j\right), k = 1,2, \dots, h, j = 2, \dots, q \tag{19}$$

The computation of DnCs in DeepDenT is performed in hidden layer order and from left to right within the same hidden layer. The output of the DnC shown in dark color in Fig. 3 is h_{t-1}^q. The final output of DeepDenT can be calculated as the output of FC with the formula given in Equation (20).

$$\hat{y}_t = \sigma\left(W_{FC} h_{t-1}^{q} + b_{FC}\right) \tag{20}$$

After presenting the computational formulas of DeepDenT, the most important problem is to propose the training algorithm for this network. The training algorithm of DeepDenT based on the differential evolution optimization (DEO) method is given in steps in the following Algorithm.

Algorithm. DeepDenT's training algorithm based on the differential evolution optimization method

Step 1. The parameters of the training algorithm are determined by the user.
cn: the number of chromosomes in a population
cr: crossover ratio
$maxitr$: the number of maximum iterations
h: the number of time steps
q: the number of hidden layers
m: the number of dendrites
p: the number of inputs or features
$limit1$: The limit value for the re-starting strategy.
$limit2$: The limit value for the early stopping rule.

Step 2. Counters for restarting and early stopping strategies and iteration counter are reset ($rsc = 0$, $esc = 0, t = 0$).

Step 3. Initial gen values for chromosomes in DEO are generated. The genes of a chromosome are all the weights and biases values of a DeepDenT. The weights and biases are generated from [0,1] intervals.

$g_{i,j}^{t}$: the jth gene value of the th chromosome at time t.

$$g_{i,j}^{t} \sim Uniform(0,1) \tag{21}$$

Step 4. For each chromosome, the fitness value is calculated using Equation (22). The fitness function is the mean squared error (MSE) calculated according to the one-step forecasting performance.

$$MSE_j^t = \frac{1}{ntrain} \sum_{t=1}^{ntrain} (y_t - \hat{y}_t)^2, j = 1,2, \dots, cn \tag{22}$$

MSE_j^t is the fitness value obtained for the jth chromosome at the tth iteration.

Step 5. The iteration counter is increased $t = t + 1$.

Step 6. For all chromosomes in the population, the mutation operator is applied using the DE/rand/1 mutation strategy. In this step, the mutation strategy is applied as in the Storn and Price (1997) study where the differential evolution algorithm was first proposed.

Step 7. For all chromosomes in the population, crossover operations are performed with the method given by Storn and Price (1997). The population is updated by comparing the candidate chromosomes obtained as a result of the crossover with the

existing chromosomes in the population.

Step 8. The chromosome with the best fitness value in the population is saved as "$bestcr$" and its fitness value as "$bestcrf$".

Step 9. The restarting strategy counter is increased ($rsc = rsc + 1$) and checked. If the $rsc > limit1$ then all positions and velocities are re-generated and the rsc is taken as zero but Pbest and gbest are saved in this step.

Step 10. The early stopping rule is checked. The esc counter is increased depending on the following condition.

$$esc = \begin{cases} esc + 1\,, if\ \frac{bestcrf^{(t)} - bestcrf^{(t-1)}}{bestcrf^{(t)}} < 10^{-3} \\ 0,\ \ otherwise \end{cases} \tag{23}$$

The early stopping rule is $esc > limit2$. If the rule is satisfied, the algorithm is stopped otherwise go to Step 6.

Step 10. The state of reaching the maximum number of iterations is checked. If $t > maxitr$, the algorithm is stopped, and the best solution is taken as $bestcr$. Otherwise, it returns to Step 6.

5 Applications

In the application, the performance of the proposed method is investigated for a total of 20-time series for two stock market indices in Turkey and the USA stock markets. The first time series analyzed is the S&P500 index (S&P 500 (GSPC), SNP - SNP Real-Time Price. Currency in USD) time series. The time series given in Table 4 for the opening values between 2014-2018 are randomly selected for use in the application. In the application, the lengths of the time series are taken as 250 and 500 to cover approximately 1 and 2 years. Thanks to the random selection, opening values in different periods of the year can be used as test data in the comparison.

Table 4 The random S&P500 time series observation dates and numbers

Series No	Starting date	Ending Date	Number of Observations
1	'2016-06-15'	'2018-06-08'	500
2	'2016-09-22'	'2018-09-17'	500
3	'2014-05-22'	'2016-05-16'	500
4	'2015-11-25'	'2017-11-17'	500
5	'2014-04-21'	'2016-04-13'	500
6	'2014-11-03'	'2015-10-29'	250
7	'2015-08-25'	'2016-08-19'	250
8	'2016-11-16'	'2017-11-13'	250
9	'2017-11-09'	'2018-11-06'	250
10	'2014-08-20'	'2015-08-17'	250

The performance of the proposed method is compared with some popular and recent ANN methods and some classical forecasting methods. LSTM proposed by Hocreiter and Schmidhuber (1997), Pi-sigma ANN (PSGM) proposed by Shin and Gosh (1991), and bootstrapped hybrid ANN (B-HANN) proposed in Egrioglu and Fildes (2022) are used in the comparison. As classical forecasting methods, random walk and Holt's linear trend exponential smoothing method are used.

For the all-time series used in the application, the data set is divided into three parts training validation and test data in the block structure. While the parameter estimations of the methods applied to the training data are performed, the forecasting performances are calculated on the validation and test data. In the application of all methods, possible values for hyperparameters are selected as similar and the best hyperparameter values are selected on the validation set. According to the best hyperparameter values, 30 different test set performances are obtained by training the methods with data other than the test set with 30 different random initializations. The statistics of the RMSE values obtained for the test set performance are presented in the tables.

$$RMSE = \sqrt{\frac{1}{ntest}\sum_{t=1}^{ntest}(y_t - \hat{y}_t)^2} \tag{24}$$

The RMSE statistics for the test data forecasting performance obtained for the time series given in Table 4 are given in Table 5 and the best hyperparameter values of all methods are given in Table 6.

Table 5 Statistics of RMSEs calculated for the test set from S&P500 time series

Random Data	Methods	Mean (Std. Dev.)	Random Walk	Holt Linear Trend
1	**LSTM**	26.7972 (6.9135)	22.2968	22.1580
	PSGM	22.2869 (0.0223)		
	B-HANN	22.1675 (0.0723)		
	DeepDenT	**14.1393 (0.1055)**		
2	**LSTM**	13.5936 (1.6621)	13.1351	12.9065
	PSGM	12.9026 (0.0174)		
	B-HANN	12.8985 (0.0534)		
	DeepDenT	**10.8420 (0.0813)**		
3	**LSTM**	11.7483 (0.0386)	11.6595	11.6601
	PSGM	11.8691 (0.1516)		
	B-HANN	11.6572 (0.0264)		
	DeepDenT	**11.0319 (0.0806)**		
4	**LSTM**	7.1527 (2.4158)	6.5195	6.1760
	PSGM	6.1906 (0.0423)		
	B-HANN	6.2591 (0.0257)		
	DeepDenT	**6.1442 (0.0336)**		
5	**LSTM**	16.9118 (0.0342)	16.8722	16.8789
	PSGM	17.1040 (0.1728)		
	B-HANN	16.8555 (0.0495)		
	DeepDenT	**12.4749 (0.0512)**		
6	**LSTM**	28.9443 (0.1245)	27.6093	27.6125
	PSGM	27.2990 (0.1948)		
	B-HANN	27.6055 (0.0353)		
	DeepDenT	**17.0242 (0.6046)**		
7	**LSTM**	16.1913 (0.2616)	15.5547	15.5074
	PSGM	15.5377 (0.0334)		

	B-HANN	15.5014 (0.0157)		
	DeepDenT	**6.1011 (0.0537)**		
8	**LSTM**	7.8493 (0.5897)	6.4085	6.1294
	PSGM	**5.9579 (0.0434)**		
	B-HANN	6.1072 (0.0220)		
	DeepDenT	6.6935 (0.0216)		
9	**LSTM**	28.5453 (1.0378)	25.0223	25.1468
	PSGM	**25.2009 (0.0157)**		
	B-HANN	25.2341 (0.0622)		
	DeepDenT	37.4449 (0.5245)		
10	**LSTM**	13.7945 (0.4205)	13.5480	13.5711
	PSGM	13.6108 (0.0408)		
	B-HANN	13.5670 (0.0084)		
	DeepDenT	**12.4463 (0.0934)**		

Table 5 shows that DeepDenT has a superior forecasting performance compared to all other methods by having a lower average RMSE value in 8 out of 10 time series, i.e. 80%. In particular, it can produce RMSE results with both lower mean and lower standard deviation for the test set than the LSTM method, which is the most popular deep ANN for the forecasting problem. It can be concluded that DeepDenT is more successful than all other methods for S&P500 time series and should be preferred.

Table 6 Best hyperparameter values of the methods applied for &P500 time series

Random Data	Methods	Number of Inputs	Number of Hidden Layer Nodes	Number of Hidden Layers	Number of the Dendrites
1	**LSTM**	1	5	1	-
	PSGM	1	3	1	-
	B-HANN	1	4	1	-
	DeepDenT	4	2	1	4
2	**LSTM**	2	2	1	-
	PSGM	1	4	1	-
	B-HANN	2	4	1	-
	DeepDenT	4	1	1	1
3	**LSTM**	1	1	1	-
	PSGM	5	4	1	-
	B-HANN	2	3	1	-
	DeepDenT	3	2	1	3
4	**LSTM**	5	2	1	-
	PSGM	5	3	1	-
	B-HANN	2	3	1	-
	DeepDenT	4	2	1	1
5	**LSTM**	1	1	1	-

	PSGM	4	3	1	-
	B-HANN	4	4	1	-
	DeepDenT	2	1	1	3
6	**LSTM**	3	1	1	-
	PSGM	2	2	1	-
	B-HANN	4	2	1	-
	DeepDenT	4	1	1	1
7	**LSTM**	2	3	1	-
	PSGM	1	5	1	-
	B-HANN	1	1	1	-
	DeepDenT	3	2	1	1
8	**LSTM**	3	4	1	-
	PSGM	4	5	1	-
	B-HANN	5	2	1	-
	DeepDenT	5	1	1	5
9	**LSTM**	2	2	1	-
	PSGM	1	4	1	-
	B-HANN	5	1	1	-
	DeepDenT	5	1	1	3
10	**LSTM**	1	1	1	-
	PSGM	1	5	1	-
	B-HANN	3	1	1	-
	DeepDenT	4	2	1	4

The second application is carried out over the opening values of the Borsa Istanbul 100 index (BIST100) between 01/02/2014 and 09/02/2018. The random series obtained for BIST100 are given in Table 7.

Table 7 The random BIST100 time series observation dates and numbers

Series No	Starting date	Ending Date	Number of Observations
1	'2014-01-31'	'2015-12-31'	500
2	'2015-11-12'	'2017-10-11'	500
3	'2016-01-20'	'2017-12-19'	500
4	'2015-06-30'	'2017-05-29'	500
5	'2015-09-01'	'2017-07-31'	500
6	'2016-05-06'	'2017-04-20'	250
7	'2015-03-31'	'2016-03-14'	250
8	'2016-01-27'	'2017-01-10'	250
9	'2014-07-18'	'2015-07-02'	250
10	'2016-03-24'	'2017-03-08'	250

The RMSE statistics for the test data forecasting performance obtained for the time series given in Table 7 are given in Table 8 and the best hyperparameter values of all methods are given in Table 8.

Table 8 Statistics of RMSEs calculated for the test set from the BIST100 time series

Random Data	Methods	Mean (Std. Dev.)	Random Walk	Holt Linear Trend
1	**LSTM**	1223.6420 (15.1430)	1207.08	1210.35
	PSGM	1214.5321 (8.1635)		
	B-HANN	1212.3883 (1.8548)		
	DeepDenT	**1181.8496 (5.5500)**		
2	**LSTM**	1151.4959 (14.1011)	**1137.14**	1141.48
	PSGM	1168.0599 (12.4945)		
	B-HANN	1142.9581 (3.0053)		
	DeepDenT	1343.9269 (9.2532)		
3	**LSTM**	1270.5353 (6.8289)	1294.22	1287.21
	PSGM	1295.6320 (5.5221)		
	B-HANN	1288.6971 (2.4888)		
	DeepDenT	**1183.4846 (2.5291)**		
4	**LSTM**	922.5592 (31.0753)	761.156	758.495
	PSGM	773.3503 (6.9704)		
	B-HANN	759.4848 (3.6537)		
	DeepDenT	**755.2842 (6.0813)**		
5	**LSTM**	1010.8165 (28.5684)	725.734	716.068
	PSGM	713.0735 (2.2901)		
	B-HANN	**711.9066 (4.4957)**		
	DeepDenT	831.7334 (7.4913)		
6	**LSTM**	795.0412 (12.7961)	761.709	760.249
	PSGM	**740.9550 (9.2555)**		
	B-HANN	759.6025 (1.3078)		
	DeepDenT	829.7748 (2.4288)		
7	**LSTM**	939.8822 (24.2007)	**852.025**	867.407
	PSGM	895.2005 (2.0108)		
	B-HANN	869.7582 (4.9018)		
	DeepDenT	895.3056 (11.7947)		
8	**LSTM**	860.0633 (33.0192)	729.921	729.921
	PSGM	746.2749 (6.3012)		
	B-HANN	731.1404 (2.4780)		
	DeepDenT	**492.0806 (4.0767)**		
9	**LSTM**	1238.0248 (36.2521)	**1181.71**	1183.01
	PSGM	1183.3946 (20.1003)		
	B-HANN	1182.7036 (2.9808)		
	DeepDenT	1348.8000 (16.9031)		
10	**LSTM**	1098.7635 (35.4691)	894.523	902.259
	PSGM	951.8524 (4.3548)		

B-HANN	904.0714 (9.5661)
DeepDenT	**790.5405 (8.4132)**

Table 8 shows that DeepDenT has a superior forecasting performance compared to all other methods by having a lower average RMSE value in 5 out of 10 time series, i.e. 50%. In particular, it can produce RMSE results with both lower mean and lower standard deviation for the test set than the LSTM method, which is the most popular deep ANN for the forecasting problem. It can be concluded that DeepDenT is more successful than all other methods for the BIST100 time series and should be preferred.

Table 9 Best hyperparameter values of the methods applied for the BIST100 time series

Random Data	Methods	Number of Inputs	Number of Hidden Layer Nodes	Number of Hidden Layers	Number of the Dendrites
1	**LSTM**	2	2	1	-
	PSGM	4	4	1	-
	B-HANN	4	5	1	-
	DeepDenT	3	1	1	3
2	**LSTM**	1	4	1	-
	PSGM	5	3	1	-
	B-HANN	5	3	1	-
	DeepDenT	5	2	1	5
3	**LSTM**	4	2	1	-
	PSGM	2	5	1	-
	B-HANN	5	1	1	-
	DeepDenT	2	1	1	1
4	**LSTM**	1	5	1	-
	PSGM	5	5	1	-
	B-HANN	4	3	1	-
	DeepDenT	4	2	1	2
5	**LSTM**	1	5	1	-
	PSGM	5	3	1	-
	B-HANN	2	2	1	-
	DeepDenT	5	2	1	2
6	**LSTM**	1	2	1	-
	PSGM	3	5	1	-
	B-HANN	2	1	1	-
	DeepDenT	4	1	1	1
7	**LSTM**	2	2	1	-
	PSGM	1	5	1	-
	B-HANN	3	3	1	-
	DeepDenT	5	1	1	5
8	**LSTM**	4	5	1	-

	PSGM	2	5	1	-
	B-HANN	5	1	1	-
	DeepDenT	5	1	1	3
9	**LSTM**	5	2	1	-
	PSGM	5	3	1	-
	B-HANN	3	5	1	-
	DeepDenT	5	2	1	3
10	**LSTM**	1	1	1	-
	PSGM	2	4	1	-
	B-HANN	4	2	1	-
	DeepDenT	4	2	1	2

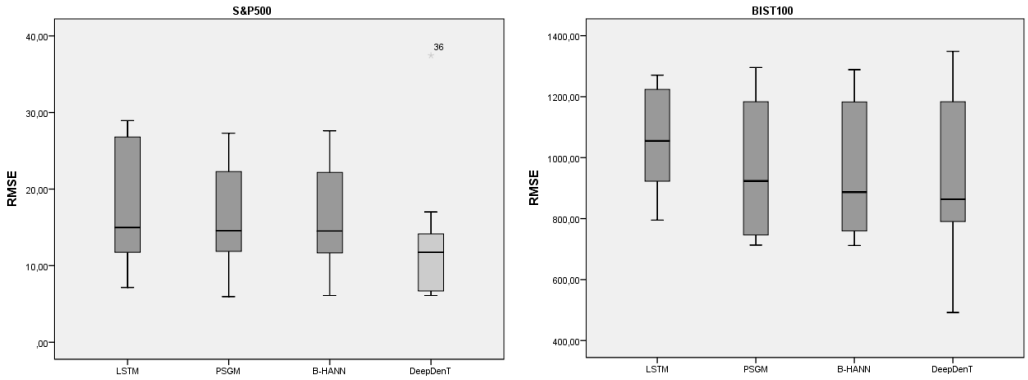

Fig. 4. Box-Plot graph of RMSE for test sets from S&P500 and BIST 100 for all methods

In Fig.4, Box-Plot Graphs of RMSE values calculated over the test set for both stock market data are given separately for all methods. As can be seen, although the results of the methods do not have a normal distribution, it is understood that DeepDenT has the lowest median.

6 Conclusion and Discussions

In this study, a new deep artificial neural network DeepDenT is proposed to solve the forecasting problem. In addition, a training algorithm based on the differential evolution algorithm is proposed for DeepDent. Since the proposed training algorithm includes a restart strategy and early stopping conditions, it can produce successful training results for DeepDent. The performance of DeepDent is investigated for 20 time series obtained from two stock exchanges. As a result of the application, it is observed that the proposed method produces successful results compared to both popular and current artificial neural networks and classical forecasting methods. The performance of the proposed new artificial neural network under training algorithms based on different artificial intelligence optimization methods is one of the topics to be investigated in the future. Another future study is to transform the proposed new neural network into a fully automatic forecasting method. For this purpose, input significance tests and different statistical tools are planned to be used for the proposed ANN.

Acknowledgment

This study is supported by the Turkish Science and Technological Researches Foundation (TUBITAK) with Award

Number:122F133, Recipient: Erol Egrioglu and Eren Bas.

Data availability Data will be made available on request.

Conflicts of interest The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

Al-Qaness MA, Ewees AA, Abualigah L, AlRassas AM, Thanh HV, Abd Elaziz M (2022) Evaluating the applications of dendritic neuron model with metaheuristic optimization algorithms for crude-oil-production forecasting. Entropy 24(11): 1674.

Bilgili M, Arslan N, Şekertekin A, Yaşar A (2022) Application of long short-term memory (LSTM) neural network based on deeplearning for electricity energy consumption forecasting. Turkish Journal of Electrical Engineering and Computer Sciences 30(1): 140-157.

Egrioglu E, Bas E (2023) Modified pi sigma artificial neural networks for forecasting. Granular Computing 8(1): 131-135.

Egrioglu E, Bas E, Chen MY (2022) Recurrent dendritic neuron model artificial neural network for time series forecasting. Information Sciences 607: 572-584.

Egrioglu E, Bas E, Karahasan O (2023) Winsorized dendritic neuron model artificial neural network and a robust training algorithm with Tukey's biweight loss function based on particle swarm optimization. Granular Computing 8(3): 491-501

Egrioglu E, Fildes R (2022) A new bootstrapped hybrid artificial neural network approach for time series forecasting. Computational Economics, 59: 1355–1383.

Gul HH, Egrioglu E, Bas E (2023) Statistical learning algorithms for dendritic neuron model artificial neural network based on sine cosine algorithm. Information Sciences 629: 398-412.

Gurney K (1989) Learning in nets of structured hypercubes. PhD Thesis, Department of Electrical Engineering, Brunel University, Middlesex, UK.

He H, Gao S, Jin T, Sato S, Zhang X (2021) A seasonal-trend decomposition-based dendritic neuron model for financial time series prediction. Applied Soft Computing 108: 107488.

Hocreiter S, Schmidhuber J (1997) Long short-term memory. Neural Computation 9(8): 1735-1780.

Karasu S, Altan A (2022) Crude oil time series prediction model based on LSTM network with chaotic Henry gas solubility optimization. Energy 242: 122964.

Liu Z, Li W, Feng J, Zhang J (2022) Research on satellite network traffic prediction based on improved GRU neural network. Sensors 22(22): 8678.

Nayak SC, Dehuri S, Cho SB (2022) Intelligent financial forecasting with an improved chemical reaction optimization algorithm based dendritic neuron model. IEEE Access 10: 130921-130943.

Ning Y, Kazemi H, Tahmasebi P (2022) A comparative machine learning study for time series oil production forecasting: ARIMA, LSTM, and Prophet. Computers Geosciences 164: 105126.

Rumelhart DE, McClelland JL (1998) PDP Research Group, Parallel distributed processing., IEEE, New York, pp 354–362.

Rumelhart E, Hinton GE, Williams RJ (1986) Learning internal representations by error propagation. Chapter 8. The M.I.T. Press, Cambridge, pp 318–362.

Sarıkaya C, Bas E, Egrioglu E (2023) Training Sigma-Pi neural networks with the grey wolf optimization algorithm. Granular Computing. https://doi.org/10.1007/s41066-023-00368-z

Shin Y, Gosh J (1991) The Pi-Sigma network: an efficient higher order neural network for pattern classification and function approximation. In: Proceedings of the International Joint Conference on Neural Networks, Seattle, pp 13-18.

Shin, Y. and Gosh, J. (1991) The Pi-sigma network: An efficient higher-order neural network for pattern classification and

function approximation. In Proceedings of the International Joint Conference on Neural Networks, pp 13-18.
Song S, Chen X, Tang C, Song S, Tang Z, Todo Y (2019) Training an approximate logic dendritic neuron model using social learning particle swarm optimization algorithm. IEEE Access 7: 141947-141959.
Storn R, Price K (1997) Differential evolution-a simple and efficient heuristic for global optimization over continuous spaces. Journal of Global Optimization 11(4): 341-359.
Tang C, Todo Y, Ji J, Lin Q, Tang Z (2021) Artificial immune system training algorithm for a dendritic neuron model. Knowledge-Based Systems 233: 107509.
Tang Y, Song Z, Zhu Y, Hou M, Tang C, Ji J (2022) Adopting a dendritic neural model for predicting stock price index movement. Expert Systems with Applications 205: 117637.
Tian C, Ma J, Zhang C, Zhan P (2018) A deep neural network model for short-term load forecast based on long short-term memory network and convolutional neural network. Energies 11(12): 3493.
Todo Y, Tamura H, Yamashita K, Tang Z (2014) Unsupervised learnable neuron model with nonlinear interaction on dendrites. Neural Networks 60: 96-103.
Veeramsetty V, Chandra DR, Salkuti SR (2021) Short-term electric power load forecasting using factor analysis and long short-term memory for smart cities. International Journal of Circuit Theory and Applications 49(6): 1678-1703.
Yadav RN, Kalra PK, John J (2007) Time series prediction with single multiplicative neuron model. Applied Soft Computing 7(4): 1157–1163.
Yilmaz A, Yolcu U (2022) Dendritic neuron model neural network trained by modified particle swarm optimization for time-series forecasting. Journal of Forecasting 41(4): 793-809.
Yilmaz A, Yolcu U (2023) A robust training of dendritic neuron model neural network for time series prediction. Neural Computing and Applications 35: 10387–10406.
Zhou T, Gao S, Wang J, Chu C, Todo Y, Tang Z (2016) Financial time series prediction using a dendritic neuron model. Knowledge-Based Systems 105: 214-224.